Week 4:

Orders and Order Details Management System

Aim

To design and implement the Orders and Order Details tables, establish relationships between them, apply constraints, insert sample data, and generate reports for order management.

Table 1: Orders

```
SQL> CREATE TABLE Orders (    Order_ID INT PRIMARY KEY,    Customer_ID INT,    Order_Date DATE,    Total_Amount DECIMAL(10,2),    Order_Status VARCH
AR(20));

Table created.
```

```
SQL> INSERT INTO Orders VALUES(101, 1, '01-SEP-2026', 2500, 'Pending');

1 row created.

SQL> INSERT INTO Orders VALUES(102, 2, '02-SEP-2026', 1800, 'Shipped');

1 row created.

SQL> INSERT INTO Orders VALUES(103, 3, '03-SEP-2026', 3200, 'Delivered');

1 row created.

SQL> INSERT INTO Orders VALUES(104, 4, '04-SEP-2026', 1500, 'Pending');

1 row created.

SQL> INSERT INTO Orders VALUES(105, 5, '05-SEP-2026', 4500, 'Delivered');

1 row created.
```

Table 2: Order Details

```
SQL> CREATE TABLE Order_Details (    Order_Detail_ID INT PRIMARY KEY,    Order_ID INT,    Product_ID INT,    Quantity INT,    Price DECIMAL(10,2),
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID));

Table created.
```

```
SQL> INSERT INTO Order_Details VALUES(1, 101, 201, 2, 500);

1 row created.

SQL> INSERT INTO Order_Details VALUES(2, 101, 202, 1, 1500);

1 row created.

SQL> INSERT INTO Order_Details VALUES(3, 102, 203, 3, 600);

1 row created.

SQL> INSERT INTO Order_Details VALUES(4, 103, 204, 2, 1600);

1 row created.

SQL> INSERT INTO Order_Details VALUES(5, 104, 205, 1, 1500);

1 row created.
```

Orders Table ,Order Details Table:

```
SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 01-SEP-26         2500 Pending
       102           2 02-SEP-26         1800 Shipped
       103           3 03-SEP-26         3200 Delivered
       104           4 04-SEP-26         1500 Pending
       105           5 05-SEP-26         4500 Delivered

SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID   ORDER_ID PRODUCT_ID   QUANTITY      PRICE
--------------- ---------- ---------- ---------- ----------
              1        101        201          2        500
              2        101        202          1       1500
              3        102        203          3        600
              4        103        204          2       1600
              5        104        205          1       1500
```

UPDATE Query

Update the order status of Order ID 101:

```
SQL> UPDATE Orders SET Order_Status = 'Delivered' WHERE Order_ID = 101;

1 row updated.
```

Update the total amount of Order ID 102:

```
SQL> UPDATE Orders SET Total_Amount = 2000 WHERE Order_ID = 102;

1 row updated.
```

Update quantity in Order_Details:

```
SQL> UPDATE Order_Details SET Quantity = 5 WHERE Order_Detail_ID = 3;

1 row updated.
```

DELETE Query

Delete a record from Order_Details:

```
SQL> DELETE FROM Order_Details WHERE Order_Detail_ID = 5;

1 row deleted.
```

Delete an order:

```
SQL> DELETE FROM Orders WHERE Order_ID = 105;

1 row deleted.
```

```
SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 01-SEP-26         2500 Delivered
       102           2 02-SEP-26         2000 Shipped
       103           3 03-SEP-26         3200 Delivered
       104           4 04-SEP-26         1500 Pending

SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID   ORDER_ID PRODUCT_ID   QUANTITY      PRICE
--------------- ---------- ---------- ---------- ----------
              1        101        201          2        500
              2        101        202          1       1500
              3        102        203          5        600
              4        103        204          2       1600
```

Reports Generated:

Report 1: Customer Order History

```
SQL> SELECT Order_ID,       Customer_ID,       Order_Date,       Total_Amount,       Order_Status FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 01-SEP-26         2500 Pending
       102           2 02-SEP-26         1800 Shipped
       103           3 03-SEP-26         3200 Delivered
       104           4 04-SEP-26         1500 Pending
       105           5 05-SEP-26         4500 Delivered
```

Report 2: Product-wise Order Report

```
SQL> SELECT Product_ID, COUNT(Order_ID) AS Times_Ordered,SUM(Quantity) AS Total_Quantity_Sold FROM Order_Details GROUP BY Product_ID;

PRODUCT_ID TIMES_ORDERED TOTAL_QUANTITY_SOLD
---------- ------------- -------------------
       205             1                   1
       201             1                   2
       202             1                   1
       203             1                   3
       204             1                   2
```

Report 3: Customer Purchase Analysis

```
SQL> SELECT Customer_ID,  COUNT(Order_ID) AS Total_Orders FROM Orders GROUP BY Customer_ID HAVING COUNT(Order_ID) =(    SELECT MAX(Order_Count)  FRO
M  (SELECT COUNT(Order_ID) AS Order_Count  FROM Orders  GROUP BY Customer_ID ));

CUSTOMER_ID TOTAL_ORDERS
----------- ------------
          1            1
          2            1
          4            1
          5            1
          3            1
```

Conclusion

The Orders and Order Details tables were successfully designed and implemented. Relationships and constraints were applied to ensure data integrity. Sample data was inserted successfully, and various reports were generated to analyze customer orders, product sales, and customer purchase patterns.